

 No description has been provided for this image

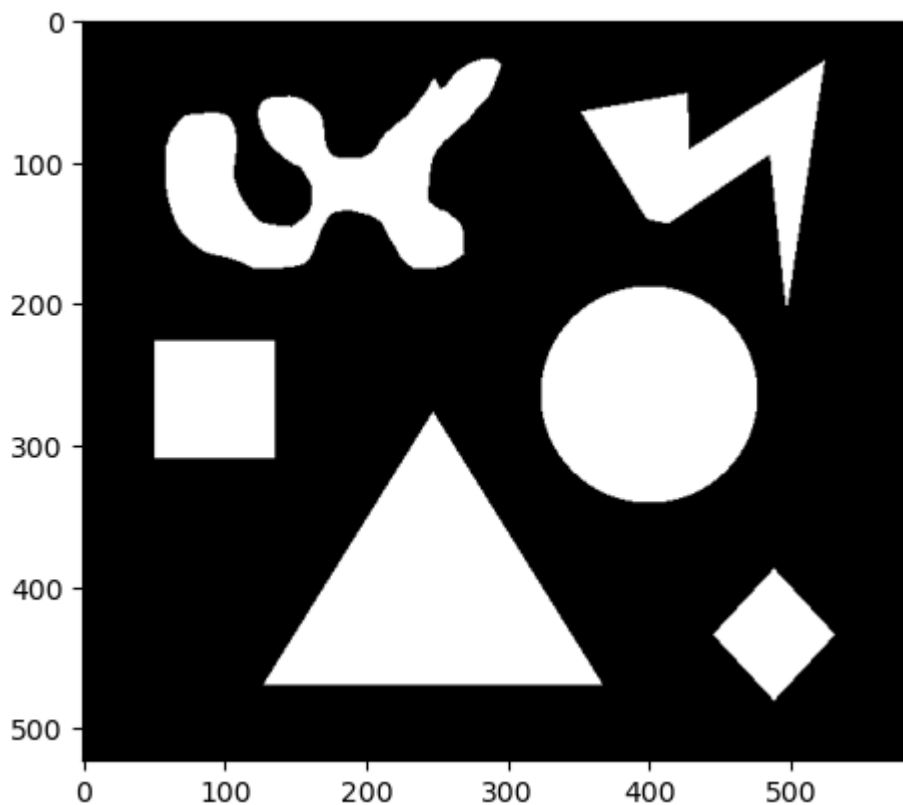
Nombre: Steven Chuiza

Fecha: 21/05/2026

Clase 3

```
In [1]: # En primer lugar, obtenemos la máscara de la imagen original mediante la umbralización
import matplotlib.pyplot as plt
import cv2
import numpy as np
img = cv2.imread('images/figuras.png')
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(img_gray, 0, 1, cv2.THRESH_OTSU)

plt.imshow(mask, cmap='gray')
plt.show()
```



```
In [2]: # Convertimos la imagen binaria en una imagen de etiquetas para tener acceso a los objetos
from skimage.measure import label

lab, num = label(mask, return_num=True)

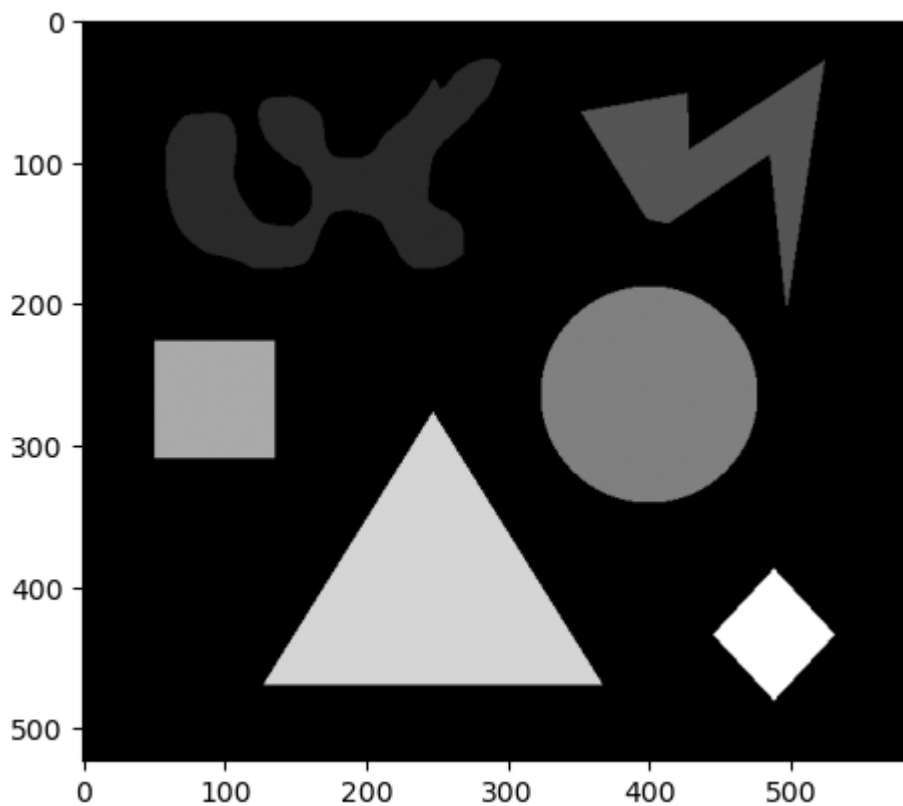
# Importante:
# En 'num' devuelve 6 valores correspondientes a los 6 objetos que hay en la imagen
# Sin embargo, 'lab' contiene hasta 7 valores diferentes, ya que también tiene el fondo
print('Número de objetos: ', num)
v, c = np.unique(lab, return_counts=True)
```

```
print('posibles valores de intensidad: ', v)

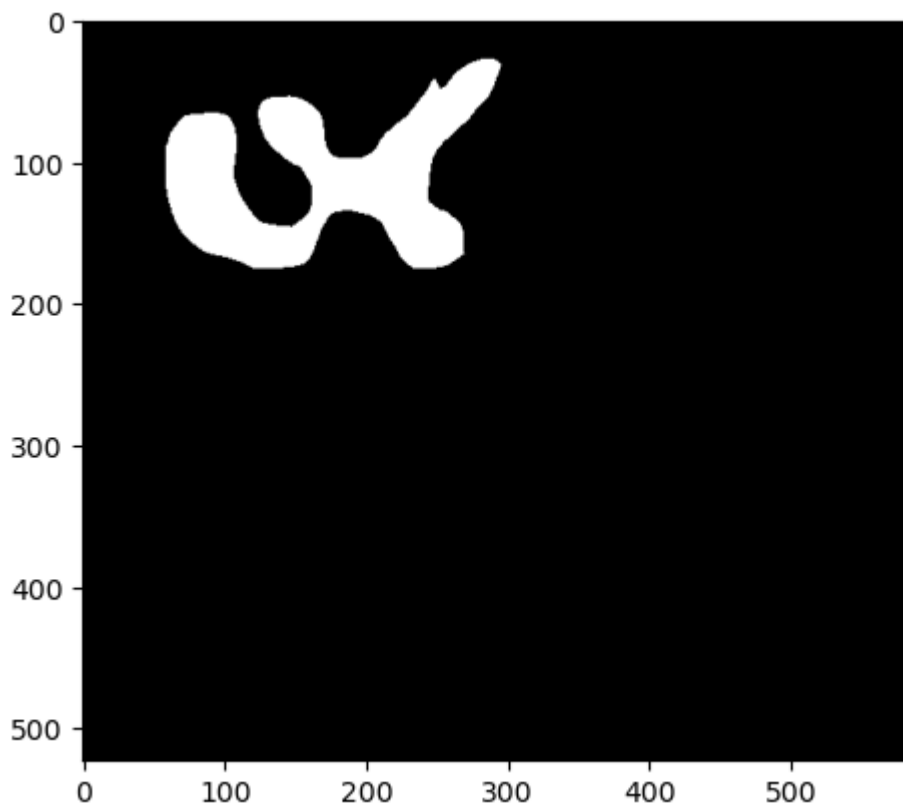
plt.imshow(lab, cmap='gray')
plt.show()
```

número de objetos: 6

posibles valores de intensidad: [0 1 2 3 4 5 6]



```
In [7]: # Podemos acceder de manera independiente al objeto que nos interese a partir de
objeto = lab == 1
objeto = objeto.astype('uint8')
plt.imshow(objeto, cmap='gray')
plt.show()
```

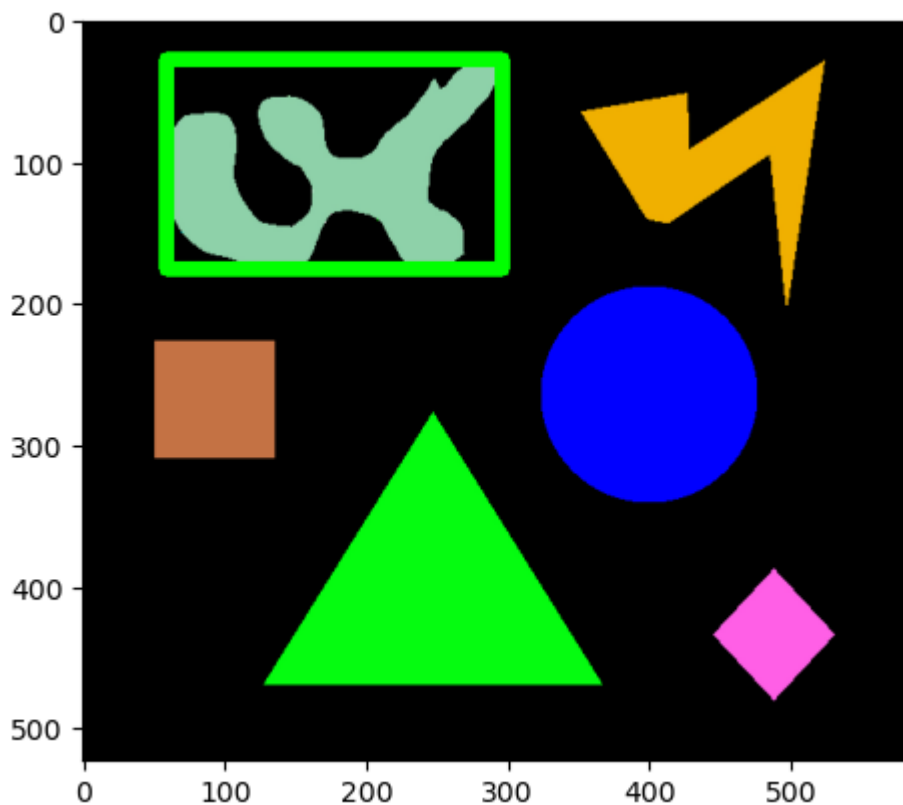


```
In [8]: from skimage.measure import regionprops
# https://scikit-image.org/docs/dev/api/skimage.measure.html#skimage.measure.regionprops

# Después, podemos extraer la bounding box de dicho objeto
prop = regionprops(objeto)
bbox = prop[0].bbox
print(bbox)
start_point = (bbox[1], bbox[0])
end_point = (bbox[3], bbox[2])
window = cv2.rectangle(img.copy(), start_point, end_point, (0, 255, 0), 10)

plt.imshow(window, cmap='gray')
plt.show()
```

(28, 59, 176, 296)

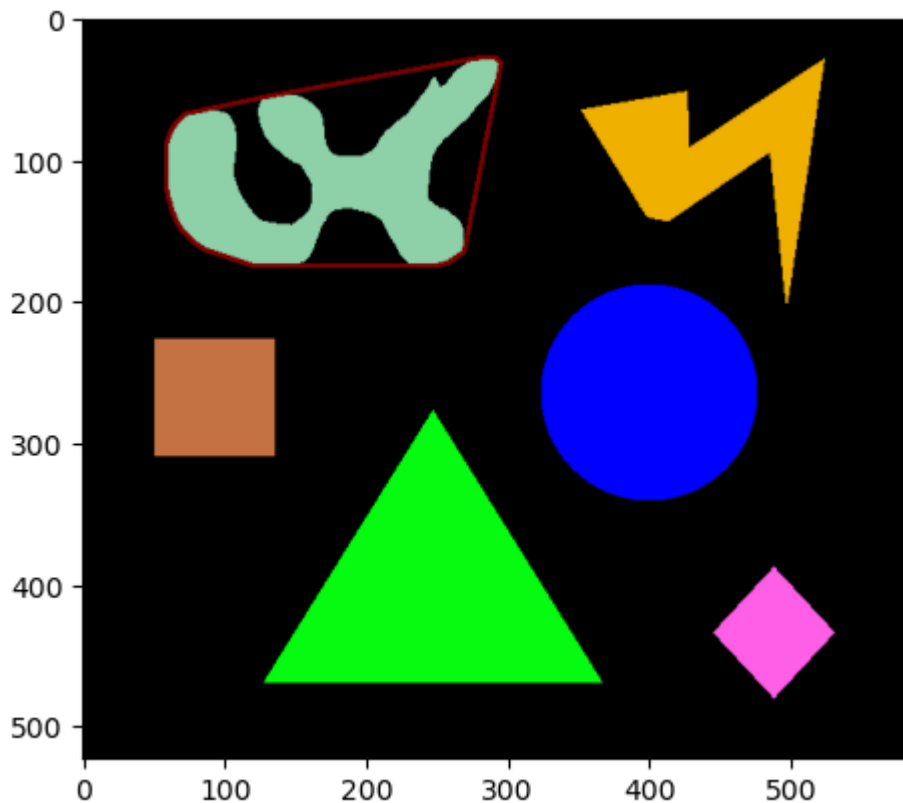


```
In [11]: # Y la convex hull image
from skimage.morphology import convex_hull_image

convex_image = convex_hull_image(objeto)
convex_image = convex_image.astype('uint8')
# plt.imshow(convex_image, cmap='gray')
# plt.show()

conts,_ = cv2.findContours(convex_image, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
imagen = cv2.drawContours(img.copy(), conts, -1, (125,0,0), 2) # Dibujamos los c

plt.imshow(imagen, cmap='gray')
plt.show()
```

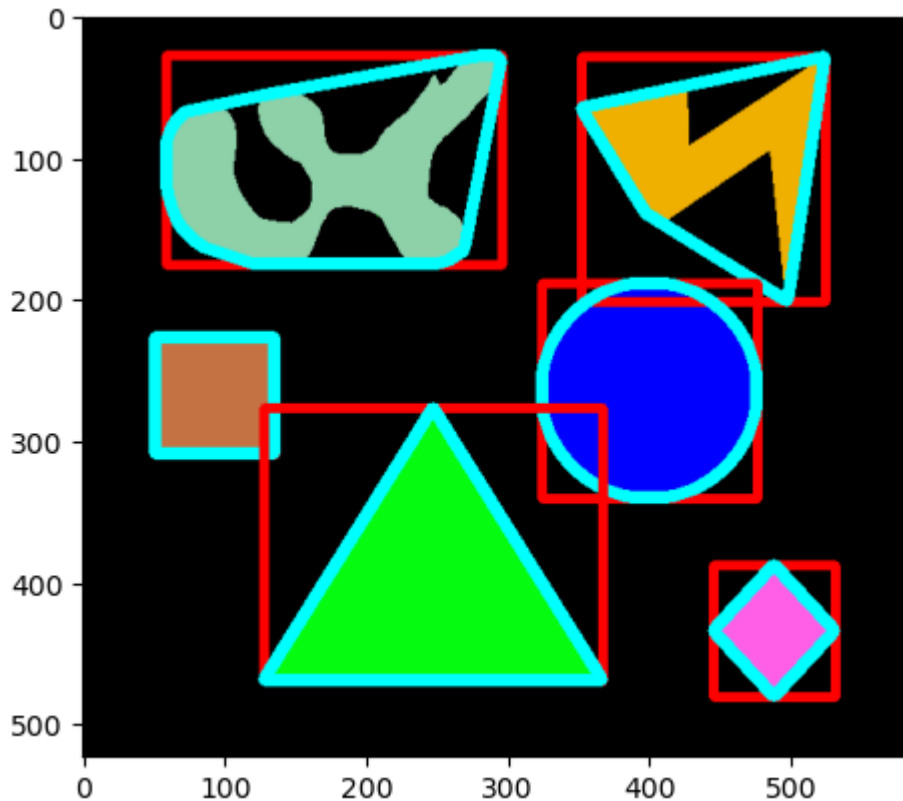


```
In [12]: # Repetimos el proceso completo para todos los objetos de la imagen
imagen = img.copy()
for i in range(1, num+1):
    objeto = lab == i
    objeto = objeto.astype('uint8')

    # bbox
    prop = regionprops(objeto)
    bbox = prop[0].bbox
    imagen = cv2.rectangle(imagen, (bbox[1], bbox[0]), (bbox[3], bbox[2]), (255,

    # convex hull
    convex_image = convex_hull_image(objeto)
    convex_image = convex_image.astype('uint8')
    conts,_ = cv2.findContours(convex_image, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIM
    imagen = cv2.drawContours(imagen, conts, -1, (0,255,255), 8)

plt.imshow(imagen, cmap='gray')
plt.show()
```



```
In [13]: # Vamos a extraer diferentes descriptores geométricos que nos ayudarán a caracte
# pip install tabulate
from tabulate import tabulate
import math

new_lab, new_num = label(mask, return_num=True)

# Extraemos las características geométricas
headers = ['', 'Raro', 'Rayo', 'Círculo', 'Cuadrado', 'Triángulo', 'Rombo']
A, BB, CA, E, ED, EX, MA, MiA, OR, P, S, CO, R = ['area'], ['bbox_area'], ['convex_area'], [
['extent'], ['major_axis'], ['minor_axis'], ['orientation'], ['perimeter'], ['so

for i in range(1, new_num+1):
    objeto = new_lab == i
    prop = regionprops(objeto.astype(np.uint8))

    A.append(np.round(prop[0].area, 4))
    BB.append(np.round(prop[0].bbox_area, 4))
    CA.append(np.round(prop[0].convex_area, 4))
    E.append(np.round(prop[0].eccentricity, 4))
    ED.append(np.round(prop[0].equivalent_diameter, 4))
    EX.append(np.round(prop[0].extent, 4))
    MA.append(np.round(prop[0].major_axis_length, 4))
    MiA.append(np.round(prop[0].minor_axis_length, 4))
    OR.append(np.round(prop[0].orientation, 4))
    P.append(np.round(prop[0].perimeter, 4))
    S.append(np.round(prop[0].solidity, 4))
    CO.append(np.round(4*math.pi*prop[0].area/prop[0].perimeter**2, 4))
    R.append(np.round(prop[0].area/prop[0].bbox_area, 4))

my_data = [tuple(A), tuple(BB), tuple(CA), tuple(E), tuple(ED), tuple(EX), \
            tuple(MA), tuple(MiA), tuple(OR), tuple(P), tuple(S), tuple(CO), tuple
```

```
print(tabulate(my_data, headers=headers))
```

	Raro	Rayo	Círculo	Cuadrado	Triángulo	
Rombo						
area	16198	10282	18344	7055	23119	4018
bbox_area	35076	29756	23104	7055	46127	7998
convex_area	27405	16959	18488	7055	23307	4102
eccentricity	0.8556	0.7941	0	0.2157	0.371	
0.3866						
equiv_diameter	143.61	114.418	152.828	94.7771	171.569	7
1.5254						
extent	0.4618	0.3455	0.794	1	0.5012	
0.5024						
major_axis	268.45	186.995	152.828	98.1428	195.774	7
6.1969						
minor_axis	138.975	113.644	152.828	95.8332	181.801	7
0.2735						
orientation	-1.4236	1.5298	-0.7854	1.5708	-1.5708	0
perimeter	1029.77	736.02	503.245	332	717.962	25
5.588						
solidity	0.5911	0.6063	0.9922	1	0.9919	
0.9795						
compactness	0.1919	0.2385	0.9102	0.8043	0.5636	
0.7729						
rectangularity	0.4618	0.3455	0.794	1	0.5012	
0.5024						

C:\Users\Admin\AppData\Local\Temp\ipykernel_6124\4149012655.py:18: FutureWarning: `RegionProperties.bbox\_area` is deprecated starting in version 0.26 and will be removed in version 2.0. Use `RegionProperties.area\_bbox` instead.

```
BB.append(np.round(prop[0].bbox_area, 4))
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_6124\4149012655.py:19: FutureWarning: `RegionProperties.convex\_area` is deprecated starting in version 0.26 and will be removed in version 2.0. Use `RegionProperties.area\_convex` instead.

```
CA.append(np.round(prop[0].convex_area, 4))
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_6124\4149012655.py:21: FutureWarning: `RegionProperties.equivalent\_diameter` is deprecated starting in version 0.26 and will be removed in version 2.0. Use `RegionProperties.equivalent\_diameter\_area` instead.

```
ED.append(np.round(prop[0].equivalent_diameter, 4))
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_6124\4149012655.py:23: FutureWarning: `RegionProperties.major\_axis\_length` is deprecated starting in version 0.26 and will be removed in version 2.0. Use `RegionProperties.axis\_major\_length` instead.

```
MA.append(np.round(prop[0].major_axis_length, 4))
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_6124\4149012655.py:24: FutureWarning: `RegionProperties.minor\_axis\_length` is deprecated starting in version 0.26 and will be removed in version 2.0. Use `RegionProperties.axis\_minor\_length` instead.

```
MiA.append(np.round(prop[0].minor_axis_length, 4))
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_6124\4149012655.py:29: FutureWarning: `RegionProperties.bbox\_area` is deprecated starting in version 0.26 and will be removed in version 2.0. Use `RegionProperties.area\_bbox` instead.

```
R.append(np.round(prop[0].area/prop[0].bbox_area, 4))
```

Repositorio de GitHub: https://github.com/tiven29/Fundamentos_Machine_Learning

In []: